

채용 리포트 · 주문·재고 API v2

주문·재고 API v2 · 제출 코드 dbff20e9379a5f61aca6317451221a539e53dffb

이 문서는 실행 근거를 정리한 자료이며 채용 결정은 사람이 합니다. LLM은 점수와 판정에 관여하지 않았습니다.

1 한눈 요약

37~57/100 · 20점 검토 대기



2 핵심 관측

확인된 결함

R-05 · 멍등 재전송

실패

하네스 케이스 R-05-idempotent-resend FAIL(ASSERTION): 검사 8개 중 2개 실패 · p1After.stock: 기대 1, 실제 0 · second.body.id == first.body.id: 기대 true, 실제 false

판정 조건: 같은 'Idempotency-Key'와 같은 본문으로 'POST /orders'를 두 번 순차 호출하면 두 번 모두 201이고 두 응답의 'id'가 같다. 'GET /products/:id'의 stock은 quantity만큼 한 번만 줄어 있으며, 첫 응답의 id로 'GET /orders/:id'를 호출하면 200이다. 두 번째 호출이 새 주문을 만들지 않았어야 한다(응답 id가 다르면 FAIL).

R-05 · 멍등 재전송 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-05>)

R-05 재생 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-05&run=9dc6589f-002c-4a44-837d-ad8c00ab8253>)

src/app.ts:23-31 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>)

src/app.ts:43-69 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>)

src/app.ts:33-41 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L33-L41>)

src/data.ts:42-53 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/data.ts#L42-L53>)

R-04 · 입력 검증

실패

하네스 케이스 R-04-validation FAIL(ASSERTION): 검사 23개 중 15개 실패 · quantityMissing.status: 기대 400, 실제 201 · quantityMissing.body.error.code: 기대 "VALIDATION_ERROR", 실제 null · quantityZero.status: 기대 400, 실제 201 · quantityZero.body.error.code: 기대 "VALIDATION_ERROR", 실제 null · quantityNegative.status: 기대 400, 실제 201 · quantityNegative.body.error.code: 기대 "VALIDATION_ERROR", 실제 null · quantityFraction.status: 기대 400, 실제 201 · quantityFraction.body.error.code: 기대 "VALIDATION_ERROR", 실제 null · quantityString.status: 기대 400, 실제 201 · quantityString.body.error.code: 기대 "VALIDATION_ERROR", 실제 null · keyMissing.status: 기대 400, 실제 201 · keyMissing.body.error.code: 기대 "VALIDATION_ERROR", 실제 null · keyEmpty.status: 기대 400, 실제 201 · keyEmpty.body.error.code: 기대 "VALIDATION_ERROR", 실제 null · p1After.stock: 기대 2, 실제 null

판정 조건: 'POST /orders'에 다음 요청을 보내면 각각 400이고 'error.code'가 'VALIDATION_ERROR'다: quantity 누락, quantity 0, quantity -1, quantity 1.5, quantity 문자열 "1", productId 누락, productId 빈 문자열, 'Idempotency-Key' 헤더 누락, 'Idempotency-Key' 빈 문자열, JSON이 아닌 본문. 모든 경우에 'GET /products/:id'의 stock이 시드 값 2 그대로다. 400으로 끝난 요청의 키는 이후 같은 키-유효한 본문으로 보내면 201로 새 주문을 만든다(멍등성 기록에 남지 않는다).

R-04 · 입력 검증 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-04>)

R-04 재생 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-04&run=c0856d5f-4471-4b54-b7a8-5220110523ca>)

src/app.ts:43-69 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>)

src/app.ts:23-31 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>)

src/app.ts:44-52 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L44-L52>)

src/app.ts:61-68 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L61-L68>)

src/data.ts:42-43 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/data.ts#L42-L43>)

확인된 강점

R-03 · 재고 부족 거절

통과

하네스 케이스 R-03-insufficient-stock PASS: 검사 9개 모두 통과

판정 조건: 초기화 후 'POST /orders'에 p1(재고 2) quantity 3, 그리고 p3(재고 0) quantity 1을 보내면 각각 409이고 'error.code'가 'INSUFFICIENT_STOCK'이다. 두 경우 모두 'GET /products/:id'의 stock이 변하지 않는다(2, 0). p1에 quantity 2(재고와 같은 수량)는 201이고 stock이 0이 된다.

R-03 · 재고 부족 거절 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-03>)

R-03 재생 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-03&run=e23d429c-8766-4c44-8f46-a9120ed1e3ae>)

src/app.ts:43-69 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>)

src/app.ts:23-31 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>)

R-08 · 다른 키 동시 요청과 재고 하한

통과

하네스 케이스 R-08-distinct-keys-concurrent PASS: 검사 7개 모두 통과

판정 조건: 초기화 후 서로 다른 'Idempotency-Key' 10개로 'POST /orders' '{ productId: "p2", quantity: 1 }' (재고 5)를 동시에 보내면 201이 정확히 5건, 409 'INSUFFICIENT_STOCK'이 정확히 5건이다. 완료 후 'GET /products/:id'의 stock이 0이고, 실행 도중과 완료 후 어느 관측에서도 stock이 음수가 아니다. 201을 받은 5건의 id는 모두 서로 다르고 각각 'GET /orders/:id'로 조회된다.

R-08 · 다른 키 동시 요청과 재고 하한 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-08>)

R-08 재생 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-08&run=52a00cd4-55d6-4c38-9f58-0cfcf29a5af1>)

src/app.ts:43-69 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>)

src/app.ts:23-31 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>)

src/app.ts:33-41 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L33-L41>)

R-06 · 멍등 키 충돌

실패

하네스 케이스 R-06-idempotency-conflict FAIL (ASSERTION): 검사 10개 중 5개 실패 · otherProduct.status: 기대 422, 실제 201 · otherProduct.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 null · otherQuantity.status: 기대 422, 실제 409 · otherQuantity.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 "INSUFFICIENT_STOCK" · p2After.stock: 기대 5, 실제 4

판정 조건: 같은 `Idempotency-Key`로 `POST /orders`를 먼저 `{ productId: "p1", quantity: 1 }`로 성공시킨 뒤 다른 본문(`{ productId: "p2", quantity: 1 }` 또는 quantity 2)으로 다시 보내면 422이고 `error.code`가 `IDEMPOTENCY\_CONFLICT`다. 새 주문이 만들어지지 않고(`GET /products/p2`·`GET /products/p1`의 stock이 첫 주문 직후와 같음) 첫 주문은 `GET /orders/:id`로 그대로 조회된다.

R-06 · 멍등 키 충돌 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-06>)

R-06 재생 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-06&run=d9d1a193-07f6-4cdc-9cac-ef3a1d4fe5f6>)

src/app.ts:43-69 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>)

src/app.ts:23-31 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L23-L31>)

src/app.ts:33-41 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L33-L41>)

src/app.ts:61-65 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L61-L65>)

3 역량별 관측

요구사항 이해와 구현 정확성

통과 2 · 실패 3 · 부분 0 · 미확정 0

- R-01 · 정상 주문 생성 **통과** 자동 판정
- R-02 · 상품-주문 조회 **통과** 자동 판정
- R-05 · 역등 재전송 **실패** 자동 판정
- R-06 · 역등 키 충돌 **실패** 자동 판정
- R-09 · 주문 취소와 재고 복구 **실패** 자동 판정

면접에서 확인할 질문

이 역량을 확인하는 키트 질문이 없습니다.

동시성과 데이터 정합성

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

Q7. R-08은 서로 다른 키 10건 동시 요청에서 201 5건과 409 5건, 재고 0으로 기대대로 통과했습니다. 동시에 들어온 요청들 사이에서 재고 확인과 차감이 어긋나지 않게 어떤 방식으로 처리하셨나요?

Q9. 팀 프로젝트에서 역등성 키를 도입해 같은 요청이 두 번 처리되지 않도록 설계하셨다고 하셨는데, 그때 키와 응답을 어디에 어떤 조건으로 저장하도록 정하셨나요?

설계와 변경 용이성

통과 0 · 실패 0 · 부분 0 · 미확정 1

- R-12 · 설계-변경 용이성 **미확정** 검토 대기

면접에서 확인할 질문

Q6. 가장 긴 함수가 POST /orders 콜백으로 27줄이라는 신호가 있습니다. 이 콜백 안에서 요청 파싱, 검증, 재고 차감, 역등성 처리를 어떤 기준으로 나누거나 합치셨나요?

Q11. 코드에 명시적 any가 3곳 있다는 신호가 있습니다. 이 지점들에서 타입을 any로 둔 이유와, 대신 쓸 수 있었던 타입 정의를 어떻게 판단하셨나요?

트레이드오프 판단과 확장 **과제로 관측 불가: 면접에서 확인**

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

Q8. 같은 서비스를 여러 인스턴스로 띄워 요청이 서로 다른 인스턴스로 들어오는 상황을 가정해 보겠습니다. 지금 구현에서 가장 먼저 바뀌어야 할 부분은 어디인가요?

Q12. 프로세스를 다시 시작해도 주문과 상태가 남아야 하는 상황을 가정해 보겠습니다. 지금 구현에서 가장 먼저 바뀌어야 할 부분은 어디인가요?

Q13. 이 경험에서 본인이 맡은 범위와 가장 어려웠던 기술적 결정을 설명해 주시겠어요?

문서화와 설명

통과 2 · 실패 0 · 부분 0 · 미확정 0

- R-10 · 실행 계약 충족 **통과** 자동 판정
- R-11 · README **통과** 자동 판정

면접에서 확인할 질문

이 역량을 확인하는 키트 질문이 없습니다.

경계 조건과 실패 처리

통과 2 · 실패 2 · 부분 0 · 미확정 0

- R-03 · 재고 부족 거절 **통과** 자동 판정
- R-04 · 입력 검증 **실패** 자동 판정
- R-07 · 같은 키 동시 요청 **실패** 자동 판정
- R-08 · 다른 키 동시 요청과 재고 하한 **통과** 자동 판정

면접에서 확인할 질문

Q4. R-03은 재고 부족과 재고 0, 재고와 같은 수량 주문까지 모두 기대대로 통과했습니다. 재고와 요청 수량을 비교하는 조건을 어떤 기준으로 정하셨나요?

테스트 설계

통과 0 · 실패 1 · 부분 0 · 미확정 2

- G1 · 테스트 실효성: 경계-검증 **실패** 자동 판정
- G2 · 테스트 실효성: 역등성 **미확정** 검토 대기
- G3 · 테스트 실효성: 취소 **미확정** 검토 대기

면접에서 확인할 질문

Q3. 제출한 테스트는 기존 상태에서 통과했지만 재고 부족 검사를 없앤 변형을 잡아내지 못했습니다. 재고 부족과 수량 경계를 검증하는 테스트를 어떤 입력과 기대값으로 설계하셨나요?

Q14. 북마크 API의 테스트는 어떤 입력과 기대 결과를 기준으로 작성하셨나요?

원인 분석 **과제로 관측 불가: 면접에서 확인**

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

Q1. R-05 실행 기록에서 같은 키로 두 번 보냈을 때 두 번째 응답의 id가 첫 번째와 달랐고 재고도 두 번 줄었습니다. 주문 생성 요청이 들어왔을 때 역등성 기록을 어떤 순서로 확인하고 갱신하도록 코드를 구성하셨나요?

Q2. R-06 실행 기록에서는 같은 키에 다른 본문을 보냈을 때 201과 409가 돌아왔고 재고도 함께 줄었습니다. 같은 키로 들어온 요청의 본문이 이전과 다를 때 어떤 값을 비교해 어떻게 판단하도록 작성하셨나요?

Q5. R-09 실행 기록에서는 취소 후 재고가 시드 값으로 돌아오지 않고 1에 머물렀습니다. 취소 요청을 처리할 때 주문 상태와 재고를 어떤 순서로 갱신하도록 작성하셨나요?

Q10. R-04 실행 기록에서는 수량 누락-0-음수 같은 요청이 400이 아니라 201로 처리되었습니다. 주문 생성 전에 입력을 검사하는 절차를 어떤 순서와 기준으로 두셨나요?

운영과 관측 가능성 **과제로 관측 불가: 면접에서 확인**

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

이 역량을 확인하는 키트 질문이 없습니다.

4 요구사항별 결과

기준	영역	판정	점수	검토 상태	관측
G1·테스트 실효성 : 경계-검증	테스트 실효성	실패	0/5	자동 판정	검증되지 않은 요구사항: R-03·M-01 SURVIVED(R-03): 하네스가 결함을 확인했지만 제출 테스트 4개 중 실패 0개로 모두 통과함 ·M-02 NOT_APPLICABLE(R-04): 대상 로직 없음·AST 휴리스틱 후보 없음, LLM 후보 없음
G2·테스트 실효성 : 역등성	테스트 실효성	미확정	?/5	검토 대기	유효한 변형 없음: 결함을 주입해 확인할 수 없어 검토 대기·M-03 NOT_APPLICABLE(R-05): 대상 로직 없음·AST 휴리스틱 후보 없음, LLM 후보 없음·M-04 NOT_APPLICABLE(R-06): 대상 로직 없음·AST 휴리스틱 후보 없음, LLM 후보 없음
G3·테스트 실효성 : 취소	테스트 실효성	미확정	?/5	검토 대기	유효한 변형 없음: 결함을 주입해 확인할 수 없어 검토 대기·M-05 NOT_APPLICABLE(R-09): 대상 로직 없음·AST 휴리스틱 후보 없음, LLM 후보 없음
R-01·정상 주문 생성	요구 기능	통과	8/8	자동 판정	하네스 케이스 R-01-normal-order PASS: 검사 6개 모두 통과
R-02·상품 주문 조회	요구 기능	통과	6/6	자동 판정	하네스 케이스 R-02-lookups PASS: 검사 21개 모두 통과
R-03·재고 부족 거절	경계- 실패	통과	7/7	자동 판정	하네스 케이스 R-03-insufficient-stock PASS: 검사 9개 모두 통과
R-04·입력 검증	경계- 실패	실패	0/6	자동 판정	하네스 케이스 R-04-validation FAIL(ASSERTION): 검사 23개 중 15개 실패·quantityMissing.status: 기대 400, 실제 201·quantityMissing.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityZero.status: 기대 400, 실제 201·quantityZero.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityNegative.status: 기대 400, 실제 201·quantityNegative.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityFraction.status: 기대 400, 실제 201·quantityFraction.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityString.status: 기대 400, 실제 201·quantityString.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·keyMissing.status: 기대 400, 실제 201·keyMissing.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·keyEmpty.status: 기대 400, 실제 201·keyEmpty.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·p1After.stock: 기대 2, 실제 null
R-05·역등 재전송	요구 기능	실패	0/14	자동 판정	하네스 케이스 R-05-idempotent-resend FAIL(ASSERTION): 검사 8개 중 2개 실패·p1After.stock: 기대 1, 실제 0·second.body.id == first.body.id: 기대 true, 실제 false
R-06·역등 키 충돌	요구 기능	실패	0/6	자동 판정	하네스 케이스 R-06-idempotency-conflict FAIL(ASSERTION): 검사 10개 중 5개 실패·otherProduct.status: 기대 422, 실제 201·otherProduct.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 null·otherQuantity.status: 기대 422, 실제 409·otherQuantity.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 "INSUFFICIENT_STOCK"·p2After.stock: 기대 5, 실제 4
R-07·같은 키 동시 요청	경계- 실패	실패	0/6	자동 판정	하네스 케이스 R-07-same-key-concurrent FAIL(ASSERTION): 검사 3개 중 3개 실패·p2After.stock: 기대 4, 실제 0·burst.count(status == 201): 기대 10, 실제 5·burst.allEqual(body.id): 기대 true, 실제 false
R-08·다른 키 동시 요청과 재고 하한	경계- 실패	통과	6/6	자동 판정	하네스 케이스 R-08-distinct-keys-concurrent PASS: 검사 7개 모두 통과
R-09·주문 취소와 재고 복구	요구 기능	실패	0/6	자동 판정	하네스 케이스 R-09-cancel FAIL(ASSERTION): 검사 14개 중 2개 실패·p1Cancelled.stock: 기대 2, 실제 1·p1CancelledAgain.stock: 기대 2, 실제 1
R-10·실행 계약 충족	실행 재현성· 문서	통과	6/6	자동 판정	기동 HEALTHY: /health 200 (412ms, 시도 5회)·하네스 케이스 R-10-health-and-reset PASS: 검사 13개 모두 통과
R-11·	실행	통과	4/4	자동 판정	README.md 존재 (36줄)·시작 명령 언급: 9행, 15행·PORT 언급: 12행, 15행

기준	영역	판정	점수	검토 상태	관측
README	재현성·문서				
R-12·설계·변경·용이성	설계·변경·용이성	미확정	?/10	검토 대기	사람 확인 대기: 설계 기준은 코드 근거와 함께 사람이 판정한다 (AI 초안은 4단계 T-407)

테스트 실효성

G1 경계·검증 **실패** SURVIVED 1, NOT_APPLICABLE 1

G2 멍등성 **미확정** NOT_APPLICABLE 2

G3 취소 **미확정** NOT_APPLICABLE 1

설계 검토 **AI 초안**

R-12 (a) 계층 분리: 라우팅·응답 처리는 src/app.ts, 데이터 접근은 src/data.ts로 나뉘어 있고 도메인 함수가 req/res에 의존하지 않아 부분적으로 충족한다. 다만 재고 확인·차감·취소 같은 도메인 규칙이 핸들러 콜백 안에 직접 들어 있어 HTTP 계층과 도메인 규칙이 완전히 분리되지 않았다. (b) 멍등성 저장소 추상화: 멍등성 기록 저장·조회 코드가 아예 없어 인터페이스나 별도 모듈 뒤에 있다고 볼 수 없다. (c) 변경 용이성: 오류 코드·상태 코드 매핑이 각 핸들러에 인라인으로 흩어져 있고 검증 규칙도 핸들러 안에 있어 새 오류 코드·검증 규칙 추가 시 여러 위치를 고쳐야 한다.

R-12·설계·변경·용이성 (<https://ohmyti.vercel.app/evaluations/069a839f-ad1b-43a0-8d98-80adffad73a6?criterion=R-12>)

src/app.ts:43-69 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L43-L69>)

src/app.ts:71-87 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/app.ts#L71-L87>)

src/data.ts:42-57 (<https://github.com/wanted-hack-inchyangv/gaeun-order-api/blob/dbff20e9379a5f61aca6317451221a539e53dfbf/src/data.ts#L42-L57>)

코드 신호: 소스 파일 4개·테스트 파일 1개·명시적 any 3곳

5 이력서 주장과 근거

팀 프로젝트로 주문/예약 API를 설계하며 대용량 트래픽 환경을 고려한 동시성 제어와 멍등성 설계 경험을 쌓음

확인 필요

과제 관측: R-05 하네스 케이스 R-05-idempotent-resend FAIL(ASSERTION): 검사 8개 중 2개 실패·p1After.stock: 기대 1, 실제 0·second.body.id == first.body.id: 기대 true, 실제 false

면접 질문: 팀 프로젝트에서 멍등성 키를 도입해 같은 요청이 두 번 처리되지 않도록 설계하셨다고 하셨는데, 그때 키와 응답을 어디에 어떤 조건으로 저장하도록 정하셨나요?

Node.js, Express를 이용한 REST API 설계와 구현 학습 **근거 있음**

과제 관측: R-04 하네스 케이스 R-04-validation FAIL(ASSERTION): 검사 23개 중 15개 실패·quantityMissing.status: 기대 400, 실제 201·quantityMissing.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityZero.status: 기대 400, 실제 201·quantityZero.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityNegative.status: 기대 400, 실제 201·quantityNegative.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityFraction.status: 기대 400, 실제 201·quantityFraction.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·quantityString.status: 기대 400, 실제 201·quantityString.body.error.code: 기대 "VALIDATION_ERROR", 실제 null·keyMissin...

근거: Express와 TypeScript로 만든 북마크 CRUD REST API 저장소로, README에 엔드포인트 표와 요청 예시, 인메모리 저장 방식이 정리되어 있다. (<https://github.com/wanted-hack-inchyangv/gaeun-bookmark-api>)

면접 질문: 이 경험에서 본인이 맡은 범위와 가장 어려웠던 기술적 결정을 설명해 주시겠어요?

인메모리 저장소와 간단한 테스트 코드를 포함합니다 **근거 있음**

과제 관측: G1 검증되지 않은 요구사항: R-03·M-01 SURVIVED(R-03): 하네스가 결함을 확인했지만 제출 테스트 4개 중 실패 0개로 모두 통과함·M-02 NOT_APPLICABLE(R-04): 대상 로직 없음·AST 휴리스틱 후보 없음, LLM 후보 없음
근거: 저장소의 커밋에 '북마크 API 테스트 작성', '입력값 검증 보완'이 있고 README에 Vitest·Supertest 사용이 적혀 있어 테스트 구성 여부를 확인할 수 있다. (<https://github.com/wanted-hack-inchyangv/gaeun-bookmark-api>)

면접 질문: 북마크 API의 테스트는 어떤 입력과 기대 결과를 기준으로 작성하셨나요?

기존 jQuery 기반 화면 일부를 React + TypeScript 컴포넌트로 전환 **근거 있음**

근거: React 19 + TypeScript + Vite 기반 저장소로, README에 컴포넌트 구조와 커스텀 훅(useLocalStorage) 사용이 설명되어 있다. (<https://github.com/wanted-hack-inchyangv/gaeun-todo-react>)

면접 질문: jQuery 기반 화면을 React + TypeScript 컴포넌트로 전환할 때 상태 관리와 이벤트 처리는 어떤 기준으로 옮기셨나요?

반응형 레이아웃과 크로스 브라우저 이슈를 점검하고 수정 **근거 있음**

근거: 저장소의 커밋에 '다크 모드와 반응형 스타일 적용'이 있어 반응형 스타일 작업 이력을 확인할 수 있다. (<https://github.com/wanted-hack-inchyangv/gaeun-todo-react>)

면접 질문: 크로스 브라우저 이슈를 점검할 때 어떤 브라우저와 조건을 기준으로 확인하셨나요?

6 면접 권고

Q1 실패 디브리핑 · 원인 분석 · 8분

R-05 실행 기록에서 같은 키로 두 번 보냈을 때 두 번째 응답의 id가 첫 번째와 달랐고 재고도 두 번 줄었습니다. 주문 생성 요청이 들어왔을 때 역등성 기록을 어떤 순서로 확인하고 갱신하도록 코드를 구성하셨나요?

Q2 실패 디브리핑 · 원인 분석 · 8분

R-06 실행 기록에서는 같은 키에 다른 본문을 보냈을 때 201과 409가 돌아왔고 재고도 함께 줄었습니다. 같은 키로 들어온 요청의 본문이 이전과 다를 때 어떤 값을 비교해 어떻게 판단하도록 작성하셨나요?

Q3 테스트 설계 · 테스트 설계 · 6분

제출한 테스트는 기존 상태에서 통과했지만 재고 부족 검사를 없앤 변형을 잡아내지 못했습니다. 재고 부족과 수량 경계를 검증하는 테스트를 어떤 입력과 기대값으로 설계하셨나요?

Q4 강점 확인 · 경계 조건과 실패 처리 · 6분

R-03은 재고 부족과 재고 0, 재고와 같은 수량 주문까지 모두 기대대로 통과했습니다. 재고와 요청 수량을 비교하는 조건을 어떤 기준으로 정하셨나요?

7 평가 범위와 한계

미평가 영역

- 대용량 트래픽 환경에서의 실제 운영 경험은 제출 자료로 확인할 수 없음
- 동시성 제어 구현의 세부 방식(잠금-트랜잭션 등)은 저장소 자료로 확인할 수 없음
- 협업 및 코드 리뷰 프로세스에서의 구체적 역할은 제출 자료로 확인할 수 없음
- 디자인 전공을 살린 문서화-구조화 역량의 실제 산출물 수준은 확인할 수 없음
- 백엔드 실무 경험 및 배포-운영 역량은 제출 자료로 확인할 수 없음

사람 검토 대기

G2 테스트 실효성: 역등성
G3 테스트 실효성: 취소
R-12 설계-변경 용이성

지원 범위

- 승인된 TypeScript 과제 템플릿과 고정 의존성으로만 실행합니다. 그 밖의 저장소는 미지원으로 표시하고 채점하지 않습니다.
- 판정은 과제 명세와 채점 기준에 적힌 항목만 다룹니다. 기준에 없는 역량은 이 리포트로 판단할 수 없으며 면접에서 확인해야 합니다.
- LLM은 근거 요약과 질문 문장, 이력서 연결에만 쓰고 점수와 판정에는 쓰지 않습니다.
- 이 리포트는 채용 결정을 대신하지 않습니다. 사람이 근거를 열어 확인한 뒤 판단합니다.

미확정 기준

G2 테스트 실효성: 역등성

유효한 변형 없음: 결함을 주입해 확인할 수 없어 검토 대기 · M-03 NOT_APPLICABLE(R-05): 대상 로직 없음 · AST 휴리스틱 후보 없음, LLM 후보 없음 · M-04 NOT_APPLICABLE(R-06): 대상 로직 없음 · AST 휴리스틱 후보 없음, LLM 후보 없음

G3 테스트 실효성: 취소

유효한 변형 없음: 결함을 주입해 확인할 수 없어 검토 대기 · M-05 NOT_APPLICABLE(R-09): 대상 로직 없음 · AST 휴리스틱 후보 없음, LLM 후보 없음

R-12 설계-변경 용이성

사람 확인 대기: 설계 기준은 코드 근거와 함께 사람이 판정한다 (AI 초안은 4단계 T-407)

LLM 사용 범위

REVIEW_WRITE: DONE · 모델 deepseek-flash · 프롬프트 evidence-review@v3+dd6e5d8c
CONTEXT_LINK: DONE · 모델 deepseek-flash · 프롬프트 context-link@v3+a12c3c30
INTERVIEW_KIT: DONE · 모델 deepseek-flash · 프롬프트 interview-kit@v1+9129ad0e

8 면접관 스코어카드

면접관이 기입하는 빈 양식입니다. 시스템은 값을 채우지 않습니다. 척도: 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

요구사항 이해와 구현 정확성1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

경계 조건과 실패 처리1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

동시성과 데이터 정합성1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

테스트 설계1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

설계와 변경 용이성1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

원인 분석 면접에서만 확인1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

트레이드오프 판단과 확장 면접에서만 확인1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

운영과 관측 가능성 면접에서만 확인1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

문서화와 설명1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

☐ 1☐ 2☐ 3☐ 4

메모

면접관 최종 의견

저장된 스코어카드

면접관이 기입한 기록입니다. 시스템은 값을 제안하지 않으며 평균과 합산을 내지 않습니다.

저장된 스코어카드가 없습니다.

9 감사 정보

평가 ID	069a839f-ad1b-43a0-8d98-80adffad73a6	제출 ID	6cba126e-1039-4118-9248-2734a18c780b
기준 버전	v2-be077fb44	하네스 버전	0.1.0+c40aa35430460704
환경 다이제스트	2044a2e63de05a7fa5be2f53d0693ecd2ea2b95df754926b0ee50473734c0b07	제출 SHA	dbff20e9379a5f61aca6317451221a539e53dfbf
저장소	https://github.com/wanted-hack-inchyan-gv/gaeun-order-api	평가 종료 시각	2026-09-20T01:44:33.182Z
REVIEW_WRITE 모델	deepseek-flash · 프롬프트 evidence-review@v3+dd6e5d8c · 호출 기록 2542ae74-f954-4e4f-8c82-87f92864ae7f	CONTEXT_LINK 모델	deepseek-flash · 프롬프트 context-link@v3+a12c3c30 · 호출 기록 47bb2adf-d648-4208-82a5-9f2bc0620e48
INTERVIEW_KIT 모델	deepseek-flash · 프롬프트 interview-kit@v1+9129ad0e · 호출 기록 5d80de09-ade2-41d4-a313-10b653c7f60a	사람 수정 이력	0건